The Foundations of Ecto

Ecto[3] is an Elixir library for data mapping and database queries. Ecto is language-integrated, so queries are written in a way that feels like Elixir and not a third-party syntax. There are other data access libraries in Elixir, but Ecto is by far the most popular. There are Ecto bindings for PostgresQL, MySQL, MSSQL, and SQLite3, but we'll only use PostgresQL in this book.

ActiveRecord is the most popular data access library in Ruby. ActiveRecord and Ecto have different philosophies, so let's compare them. After that, we'll take a high-level pass over Ecto's concepts.

Ecto vs. ActiveRecord

Both ActiveRecord and Ecto have a significant focus on usability, security, and flexibility. They both feel familiar and powerful after using them for only a small amount of time. But they are implemented with entirely different design patterns. These design differences make them feel like opposites.

ActiveRecord is implemented using the Active Record[4] design pattern. (As a note, this name existed before the ActiveRecord library did.) This design pattern combines data access and persistence directly on the object that holds the data. Everything you do in ActiveRecord happens on the model class directly. Queries, preloads, updates, inserts, and the like all originate from your model classes. If we wanted to fetch and update a `Person` record, it would look like this:

```
irb> steve = MyApp.Person.find_by(name: "Steve")
irb> steve.update!(name: "Stephen")
```

Ecto is implemented using the Repository[5] design pattern. This pattern requires that all data mapping and query operations happen through a centralized Repository. This creates a different programming experience. Let's take a look at the previous example, implemented using Ecto:

```
iex> steve = Repo.get_by(MyApp.Person, name: "Steve")
iex> changes = Ecto.Changeset.change(steve, name: "Stephen")
iex> Repo.update!(changes)
```

"Magic" is a term that's often used when talking about data access libraries. There will always be some magic when translating between a database and an application—the more removed from the database the app is, the more

3. https://github.com/elixir-ecto/ecto
4. https://www.martinfowler.com/eaaCatalog/activeRecord.html
5. https://martinfowler.com/eaaCatalog/repository.html